

Upload and Retrieve Data with Viam's Data Client API

 [Edit](#)

The data client API allows you to upload and retrieve data to and from the Viam Cloud.

The data client API supports the following methods:

Methods to upload data like images or sensor readings directly to the [Viam app](#):

Method Name	Description
BinaryDataCaptureUpload	Upload binary data collected on your machine through a specific component and the relevant metadata to the Viam app.
TabularDataCaptureUpload	Upload tabular data collected on your machine through a specific component to the Viam app.
FileUpload	Upload arbitrary files stored on your machine to the Viam app by file name.
FileUploadFromPath	Upload files stored on your machine to the Viam app by filepath.
StreamingDataCaptureUpload	Upload the contents of streaming binary data and the relevant metadata to the Viam app.

Methods to download, filter, tag, or perform other tasks on data like images or sensor readings:

Method Name	Description
GetLatestTabularData	Gets the most recent tabular data captured from the specified data source, as long as it was synced within the last year.
ExportTabularData	Obtain unified tabular data and metadata from the specified data source.
TabularDataByFilter	Retrieve optionally filtered tabular data from the Viam app.
TabularDataBySQL	Obtain unified tabular data and metadata, queried with SQL. Make sure your API key has permissions at the organization level in order to use this.
TabularDataByMQL	Obtain unified tabular data and metadata, queried with MQL.
BinaryDataByFilter	Retrieve optionally filtered binary data from the Viam app.
BinaryDataByIDs	Retrieve binary data from the Viam app by <code>BinaryID</code> .
DeleteTabularData	Delete tabular data older than a specified number of days.
DeleteBinaryDataByFilter	Filter and delete binary data.
DeleteBinaryDataByIDs	Filter and delete binary data by ids.
AddTagsToBinaryDataByIDs	Add tags to binary data by ids.
AddTagsToBinaryDataByFilter	Add tags to binary data by filter.
RemoveTagsFromBinaryDataByIDs	Remove tags from binary by ids.
RemoveTagsFromBinaryDataByFilter	Remove tags from binary data by filter.

Method Name	Description
TagsByFilter	Get a list of tags using a filter.
AddBoundingBoxToImageByID	Add a bounding box to an image specified by its BinaryID.
RemoveBoundingBoxFromImageByID	Removes a bounding box from an image specified by its BinaryID.
BoundingBoxLabelsByFilter	Get a list of bounding box labels using a Filter.
GetDatabaseConnection	Get a connection to access a MongoDB Atlas Data federation instance.
ConfigureDatabaseUser	Configure a database user for the Viam organization's MongoDB Atlas Data Federation instance.
AddBinaryDataToDatasetByIDs	Add the <code>BinaryData</code> to the provided dataset.
RemoveBinaryDataFromDatasetByIDs	Remove the BinaryData from the provided dataset.

Methods to work with datasets:

Method Name	Description
CreateDataset	Create a new dataset.
DeleteDataset	Delete a dataset.
RenameDataset	Rename a dataset specified by the dataset ID.
ListDatasetsByOrganizationID	Get the datasets in an organization.
ListDatasetsByIDs	Get a list of datasets using their IDs.

Establish a connection

To use the Viam data client API, you first need to instantiate a `ViamClient` and then instantiate a `DataClient` .

You will also need an API key and API key ID to authenticate your session. To get an API key (and corresponding ID), you have two options:

- [Create an API key using the Viam app](#)
- [Create an API key using the Viam CLI](#)

The following example instantiates a `ViamClient` , authenticating with an API key, and then instantiates a `DataClient` :

Python

TypeScript

```

1  import asyncio
2
3  from viam.rpc.dial import DialOptions, Credentials
4  from viam.app.viam_client import ViamClient
5
6
7  async def connect() -> ViamClient:
8      dial_options = DialOptions(
9          credentials=Credentials(
10             type="api-key",
11             # Replace "<API-KEY>" (including brackets) with your machine's API key
12             payload='<API-KEY>',
13         ),
14         # Replace "<API-KEY-ID>" (including brackets) with your machine's
15         # API key ID
16         auth_entity='<API-KEY-ID>'
17     )
18     return await ViamClient.create_from_dial_options(dial_options)
19
20
21  async def main():
22      # Make a ViamClient
23      viam_client = await connect()
24      # Instantiate a DataClient to run data client API methods on
25      data_client = viam_client.data_client
26
27      viam_client.close()
28
29  if __name__ == '__main__':
30      asyncio.run(main())

```

Once you have instantiated a `DataClient`, you can run [API methods](#) against the `DataClient` object (named `data_client` in the examples).

API

BinaryDataCaptureUpload

Upload binary data collected on your machine through a specific component and the relevant metadata to the [Viam app](#). Uploaded binary data can be found under the **Images**, **Point clouds**, or **Files** subtab of the app's [Data tab](#), depending on the type of data that you upload.

[Python](#) [Flutter](#)

Parameters:

- `binary_data` ([bytes](#)) (required): The data to be uploaded, represented in bytes.
- `part_id` ([str](#)) (required): Part ID of the component used to capture the data.
- `component_type` ([str](#)) (required): Type of the component used to capture the data (for example, "movement_sensor").
- `component_name` ([str](#)) (required): Name of the component used to capture the data.
- `method_name` ([str](#)) (required): Name of the method used to capture the data.
- `file_extension` ([str](#)) (required): The file extension of binary data including the period, for example .jpg, .png, .pcd. The backend will route the binary to its corresponding mime type based on this extension. Files with a .jpeg, .jpg, or .png extension will be saved to the images tab.
- `method_parameters` (Mapping[[str](#), Any]) (optional): Optional dictionary of method parameters. No longer in active use.
- `tags` (List[[str](#)]) (optional): Optional list of tags to allow for tag-based data filtering when retrieving data.
- `data_request_times` (Tuple[[datetime.datetime](#), [datetime.datetime](#)]) (optional): Optional tuple containing datetime objects denoting the times this data was requested[0] by the robot and received[1] from the appropriate sensor.

Returns:

- ([str](#)): The `binary_data_id` of the uploaded data.

Raises:

- (`GRPCError`): If an invalid part ID is passed.

Example:

```
1  time_requested = datetime(2023, 6, 5, 11)
2  time_received = datetime(2023, 6, 5, 11, 0, 3)
3
4  file_id = await data_client.binary_data_capture_upload(
5      part_id="INSERT YOUR PART ID",
6      component_type='camera',
7      component_name='my_camera',
8      method_name='GetImages',
9      method_parameters=None,
10     tags=["tag_1", "tag_2"],
11     data_request_times=[time_requested, time_received],
12     file_extension=".jpg",
13     binary_data=b"Encoded image bytes"
14 )
```

For more information, see the [Python SDK Docs](#).

TabularDataCaptureUpload

Upload tabular data collected on your machine through a specific `component` to the [Viam app](#). Uploaded tabular data can be found under the **Sensors** subtab of the app's [Data tab](#).

Python [Flutter](#)

Parameters:

- `tabular_data` (List[Mapping[[str](#), Any]]) (required): List of the data to be uploaded, represented tabularly as a collection of dictionaries. Must include the key “readings” for sensors.
- `part_id` ([str](#)) (required): Part ID of the component used to capture the data.
- `component_type` ([str](#)) (required): Type of the component used to capture the data (for example, “rdk:component:movement_sensor”).
- `component_name` ([str](#)) (required): Name of the component used to capture the data.
- `method_name` ([str](#)) (required): Name of the method used to capture the data.
- `data_request_times` (List[Tuple[[datetime.datetime](#), [datetime.datetime](#)]]) (required): List of tuples, each containing datetime objects denoting the times this data was requested[0] by the robot and received[1] from the appropriate sensor. Passing a list of tabular data and Timestamps with length n > 1 will result in n datapoints being uploaded, all tied to the same metadata.
- `method_parameters` (Mapping[[str](#), Any]) (optional): Optional dictionary of method parameters. No longer in active use.
- `tags` (List[[str](#)]) (optional): Optional list of tags to allow for tag-based data filtering when retrieving data.

Returns:

- ([str](#)): The file_id of the uploaded data.

Raises:

- (GRPCError): If an invalid part ID is passed.
- (ValueError): If a list of Timestamp objects is provided and its length does not match the length of the list of tabular data.

Example:

```

1  from datetime import datetime
2
3  time_requested = datetime(2023, 6, 5, 11)
4  time_received = datetime(2023, 6, 5, 11, 0, 3)
5  file_id = await data_client.tabular_data_capture_upload(
6      part_id="INSERT YOUR PART ID",
7      component_type='rdk:component:movement_sensor',
8      component_name='my_movement_sensor',
9      method_name='Readings',
10     tags=["sensor_data"],
11     data_request_times=[(time_requested, time_received)],
12     tabular_data=[{
13         'readings': {
14             'linear_velocity': {'x': 0.5, 'y': 0.0, 'z': 0.0},
15             'angular_velocity': {'x': 0.0, 'y': 0.0, 'z': 0.1}
16         }
17     }]
18 )

```

For more information, see the [Python SDK Docs](#).

FileUpload

Upload arbitrary files stored on your machine to the [Viam app](#) by file name. If uploaded with a file extension of .jpeg/.jpg/.png, uploaded files can be found in the **Images** subtab of the app's [Data tab](#). If .pcd, the uploaded files can be found in the **Point clouds** subtab. All other types of uploaded files can be found under the **Files** subtab of the app's [Data tab](#).

[Python](#) [Flutter](#)

Parameters:

- `part_id` ([str](#)) (required): Part ID of the resource associated with the file.
- `data` ([bytes](#)) (required): Bytes representing file data to upload.
- `component_type` ([str](#)) (optional): Optional type of the component associated with the file (for example, “movement_sensor”).
- `component_name` ([str](#)) (optional): Optional name of the component associated with the file.
- `method_name` ([str](#)) (optional): Optional name of the method associated with the file.
- `file_name` ([str](#)) (optional): Optional name of the file. The empty string “” will be assigned as the file name if one isn’t provided.
- `method_parameters` (Mapping[[str](#), Any]) (optional): Optional dictionary of the method parameters. No longer in active use.
- `file_extension` ([str](#)) (optional): Optional file extension. The empty string “” will be assigned as the file extension if one isn’t provided. Files with a .jpeg, .jpg, or .png extension will be saved to the images tab.
- `tags` (List[[str](#)]) (optional): Optional list of tags to allow for tag-based filtering when retrieving data.

Returns:

- ([str](#)): ID of the new file.

Raises:

- (GRPCError): If an invalid part ID is passed.

Example:

```

1  file_id = await data_client.file_upload(
2      data=b"Encoded image bytes",
3      part_id="INSERT YOUR PART ID",
4      tags=["tag_1", "tag_2"],
5      file_name="your-file",
6      file_extension=".txt"
7  )

```

For more information, see the [Python SDK Docs](#).

FileUploadFromPath

Upload files stored on your machine to the [Viam app](#) by filepath. Uploaded files can be found under the **Files** subtab of the app's [Data tab](#).

Python

Parameters:

- `filepath` ([str](#)) (required): Absolute filepath of file to be uploaded.
- `part_id` ([str](#)) (required): Part ID of the component associated with the file.
- `component_type` ([str](#)) (optional): Optional type of the component associated with the file (for example, “movement_sensor”).
- `component_name` ([str](#)) (optional): Optional name of the component associated with the file.
- `method_name` ([str](#)) (optional): Optional name of the method associated with the file.
- `method_parameters` (Mapping[[str](#), Any]) (optional): Optional dictionary of the method parameters. No longer in active use.
- `tags` (List[[str](#)]) (optional): Optional list of tags to allow for tag-based filtering when retrieving data.

Returns:

- ([str](#)): ID of the new file.

Raises:

- (GRPCError): If an invalid part ID is passed.
- (FileNotFoundError): If the provided filepath is not found.

Example:

```
1 file_id = await data_client.file_upload_from_path(  
2     part_id="INSERT YOUR PART ID",  
3     tags=["tag_1", "tag_2"],  
4     filepath="/Users/<your-username>/<your-directory>/<your-file.txt>"  
5 )
```

For more information, see the [Python SDK Docs](#).

StreamingDataCaptureUpload

Upload the contents of streaming binary data and the relevant metadata to the [Viam app](#). Uploaded streaming data can be found under the [Data tab](#).

Python

Flutter

Parameters:

- `data` ([bytes](#)) (required): the data to be uploaded.
- `part_id` ([str](#)) (required): Part ID of the resource associated with the file.
- `file_ext` ([str](#)) (required): file extension type for the data. required for determining MIME type.
- `component_type` ([str](#)) (optional): Optional type of the component associated with the file (for example, “movement_sensor”).
- `component_name` ([str](#)) (optional): Optional name of the component associated with the file.
- `method_name` ([str](#)) (optional): Optional name of the method associated with the file.
- `method_parameters` (Mapping[[str](#), Any]) (optional): Optional dictionary of the method parameters. No longer in active use.
- `data_request_times` (Tuple[[datetime.datetime](#), [datetime.datetime](#)]) (optional): Optional tuple containing datetime objects denoting the times this data was requested[0] by the robot and received[1] from the appropriate sensor.
- `tags` (List[[str](#)]) (optional): Optional list of tags to allow for tag-based filtering when retrieving data.

Returns:

- ([str](#)): The binary_data_id of the uploaded data.

Raises:

- (GRPCError): If an invalid part ID is passed.

Example:

```
1  time_requested = datetime(2023, 6, 5, 11)
2  time_received = datetime(2023, 6, 5, 11, 0, 3)
3
4  file_id = await data_client.streaming_data_capture_upload(
5      data="byte-data-to-upload",
6      part_id="INSERT YOUR PART ID",
7      file_ext="png",
8      component_type='motor',
9      component_name='left_motor',
10     method_name='IsPowered',
11     data_request_times=[time_requested, time_received],
12     tags=["tag_1", "tag_2"]
13 )
```

For more information, see the [Python SDK Docs](#).

GetLatestTabularData

Gets the most recent tabular data captured from the specified data source, as long as it was synced within the last year.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `part_id` ([str](#)) (required): The ID of the part that owns the data.
- `resource_name` ([str](#)) (required): The name of the requested resource that captured the data. Ex: “my-sensor”.
- `resource_api` ([str](#)) (required): The API of the requested resource that captured the data. Ex: “rdk:component:sensor”.
- `method_name` ([str](#)) (required): The data capture method name. Ex: “Readings”.

Returns:

- (Tuple[[datetime.datetime](#), [datetime.datetime](#), Dict[[str](#), [viam.utils.ValueTypes](#)]] | None): A return value of None means that data hasn’t been synced yet for the data source or the most recently captured data was over a year ago, otherwise the returned tuple contains the following:
 datetime: The time captured, datetime: The time synced, Dict[str, ValueTypes]: The latest tabular data captured from the specified data source.

Example:

```
1  tabular_data = await data_client.get_latest_tabular_data(
2      part_id="77ae3145-7b91-123a-a234-e567cdca8910",
3      resource_name="camera-1",
4      resource_api="rdk:component:camera",
5      method_name="GetImage"
6  )
7
8  if tabular_data:
9      time_captured, time_synced, payload = tabular_data
10     print(f"Time Captured: {time_captured}")
11     print(f"Time Synced: {time_synced}")
12     print(f"Payload: {payload}")
13 else:
14     print(f"No data returned: {tabular_data}")
```

For more information, see the [Python SDK Docs](#).

ExportTabularData

Obtain unified tabular data and metadata from the specified data source.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `part_id` ([str](#)) (required): The ID of the part that owns the data.
- `resource_name` ([str](#)) (required): The name of the requested resource that captured the data.
- `resource_api` ([str](#)) (required): The API of the requested resource that captured the data.
- `method_name` ([str](#)) (required): The data capture method name.
- `start_time` ([datetime.datetime](#)) (optional): Optional start time for requesting a specific range of data.
- `end_time` ([datetime.datetime](#)) (optional): Optional end time for requesting a specific range of data.

Returns:

- ([List\[TabularDataPoint\]](#)): The unified tabular data and metadata.

Example:

```
1  tabular_data = await data_client.export_tabular_data(  
2      part_id="<PART-ID>",  
3      resource_name="<RESOURCE-NAME>",  
4      resource_api="<RESOURCE-API>",  
5      method_name="<METHOD-NAME>",  
6      start_time="<START_TIME>"  
7      end_time="<END_TIME>"  
8  )  
9  
10 print(f"My data: {tabular_data}")
```

For more information, see the [Python SDK Docs](#).

TabularDataByFilter

Retrieve optionally filtered tabular data from the [Viam app](#). You can also find your tabular data under the **Sensors** subtab of the app's [Data tab](#).

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `filter` ([viam.proto.app.data.Filter](#)) (optional): Optional Filter specifying tabular data to retrieve. No Filter implies all tabular data.
- `limit` ([int](#)) (optional): The maximum number of entries to include in a page. Defaults to 50 if unspecified.
- `sort_order` ([viam.proto.app.data.Order.ValueType](#)) (optional): The desired sort order of the data.
- `last` ([str](#)) (optional): Optional string indicating the object identifier of the last-returned data. This object identifier is returned by calls to `TabularDataByFilter` as the last value. If provided, the server will return the next data entries after the last object identifier.
- `count_only` ([bool](#)) (required): Whether to return only the total count of entries.
- `include_internal_data` ([bool](#)) (required): Whether to return the internal data. Internal data is used for Viam-specific data ingestion, like cloud SLAM. Defaults to False.
- `dest` ([str](#)) (optional): Optional filepath for writing retrieved data.

Returns:

- (Tuple[List[TabularData], [int](#), [str](#)]): A tuple containing the following: List[TabularData]: The tabular data, int: The count (number of entries), str: The last-returned page ID.

Example:

```
1  from viam.utils import create_filter  
2  
3  my_data = []  
4  my_filter = create_filter(component_name="motor-1")  
5  last = None  
6  while True:  
7      tabular_data, count, last = await data_client.tabular_data_by_filter(my_filter, last=last)  
8      if not tabular_data:  
9          break  
10     my_data.extend(tabular_data)  
11  
12 print(f"My data: {my_data}")
```

For more information, see the [Python SDK Docs](#).

TabularDataBySQL

Obtain unified tabular data and metadata, queried with SQL. Make sure your API key has permissions at the organization level in order to use this.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `organization_id` ([str](#)) (required): The ID of the organization that owns the data. You can obtain your organization ID from the Viam app's organization settings page.
- `sql_query` ([str](#)) (required): The SQL query to run.

Returns:

- (List[Dict[[str](#), [viam.utils.ValueTypes](#) | [datetime.datetime](#)]]): An array of decoded BSON data objects.

Example:

```
1 data = await data_client.tabular_data_by_sql(  
2     organization_id="<YOUR-ORG-ID>",  
3     sql_query="SELECT * FROM readings LIMIT 5"  
4 )
```

For more information, see the [Python SDK Docs](#).

TabularDataByMQL

Obtain unified tabular data and metadata, queried with MQL.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `organization_id` ([str](#)) (required): The ID of the organization that owns the data. You can obtain your organization ID from the Viam app's organization settings page.
- `query` (List[[bytes](#)] | List[Dict[[str](#), Any]]) (required): The MQL query to run, as a list of MongoDB aggregation pipeline stages. Note: Each stage can be provided as either a dictionary or raw BSON bytes, but support for bytes will be removed in the future, so using a dictionary is preferred.
- `use_recent_data` ([bool](#)) (optional): Whether to query blob storage or your recent data store. Defaults to False.

Returns:

- (List[Dict[[str](#), [viam.utils.ValueTypes](#) | [datetime.datetime](#)]]): An array of decoded BSON data objects.

Example:

```
1 import bson  
2  
3 tabular_data = await data_client.tabular_data_by_mql(organization_id="<YOUR-ORG-ID>", query=[  
4     { '$match': { 'location_id': '<YOUR-LOCATION-ID>' } },  
5     { "$limit": 5 }  
6 ])  
7  
8 print(f"Tabular Data: {tabular_data}")
```

For more information, see the [Python SDK Docs](#).

BinaryDataByFilter

Retrieve optionally filtered binary data from the [Viam app](#). You can also find your binary data under the **Images**, **Point clouds**, or **Files** subtab of the app's [Data tab](#), depending on the type of data that you have uploaded.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `filter` ([viam.proto.app.data.Filter](#)) (optional): Optional Filter specifying tabular data to retrieve. No Filter implies all binary data.
- `limit` ([int](#)) (optional): The maximum number of entries to include in a page. Defaults to 50 if unspecified.
- `sort_order` ([viam.proto.app.data.Order.ValueType](#)) (optional): The desired sort order of the data.
- `last` ([str](#)) (optional): Optional string indicating the object identifier of the last-returned data. This object identifier is returned by calls to `BinaryDataByFilter` as the last value. If provided, the server will return the next data entries after the last object identifier.
- `include_binary_data` ([bool](#)) (required): Boolean specifying whether to actually include the binary file data with each retrieved file. Defaults to true (that is, both the files' data and metadata are returned).
- `count_only` ([bool](#)) (required): Whether to return only the total count of entries.
- `include_internal_data` ([bool](#)) (required): Whether to return the internal data. Internal data is used for Viam-specific data ingestion, like cloud SLAM. Defaults to False.
- `dest` ([str](#)) (optional): Optional filepath for writing retrieved data.

Returns:

- (Tuple[List[viam.proto.app.data.BinaryData], [int](#), [str](#)]): A tuple containing the following: List[viam.proto.app.data.BinaryData]: The binary data, int: The count (number of entries), str: The last-returned page ID.

Example:

```
1  from viam.utils import create_filter
2  from viam.proto.app.data import Filter, TagsFilter, TagsFilterType
3
4  # Get data captured from camera components
5  my_data = []
6  last = None
7  my_filter = create_filter(component_name="camera-1")
8
9  while True:
10     data, count, last = await data_client.binary_data_by_filter(
11         my_filter, limit=1, last=last)
12     if not data:
13         break
14     my_data.extend(data)
15
16  print(f"My data: {my_data}")
17
18  # Get untagged data from a dataset
19
20  my_untagged_data = []
21  last = None
22  tags_filter = TagsFilter(type=TagsFilterType.TAGS_FILTER_TYPE_UNTAGGED)
23  my_filter = Filter(
24     dataset_id="66db6fe7d93d1ade24cd1dc3",
25     tags_filter=tags_filter
26 )
27
28  while True:
29     data, count, last = await data_client.binary_data_by_filter(
30         my_filter, last=last, include_binary_data=False)
31     if not data:
32         break
33     my_untagged_data.extend(data)
```

For more information, see the [Python SDK Docs](#).

BinaryDataByIDs

Retrieve binary data from the [Viam app](#) by `BinaryID` . You can also find your binary data under the **Images**, **Point clouds**, or **Files** subtab of the app's [Data tab](#), depending on the type of data that you have uploaded.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `binary_ids` ([List\[viam.proto.app.data.BinaryID\]](#) | [List\[str\]](#)) (required): Binary data id strings specifying the desired data or BinaryID objects. Must be non-empty. Note: BinaryID objects are deprecated and will be removed in a future release. Please use the binary data id field instead.
- `dest` ([str](#)) (optional): Optional filepath for writing retrieved data.

Returns:

- ([List\[viam.proto.app.data.BinaryData\]](#)): The binary data.

Raises:

- (`GRPCError`): If no binary data id strings or BinaryID objects are provided.

Example:

```
1  binary_metadata, count, last = await data_client.binary_data_by_filter(  
2      include_binary_data=False  
3  )  
4  
5  my_ids = []  
6  
7  for obj in binary_metadata:  
8      my_ids.append(obj.metadata.binary_data_id)  
9  
10 binary_data = await data_client.binary_data_by_ids(my_ids)
```

For more information, see the [Python SDK Docs](#).

DeleteTabularData

Delete tabular data older than a specified number of days.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `organization_id` ([str](#)) (required): ID of organization to delete data from. You can obtain your organization ID from the Viam app's organization settings page.
- `delete_older_than_days` ([int](#)) (required): Delete data that was captured up to this many days ago. For example if delete_older_than_days is 10, this deletes any data that was captured up to 10 days ago. If it is 0, all existing data is deleted.

Returns:

- ([int](#)): The number of items deleted.

Example:

```
1  tabular_data = await data_client.delete_tabular_data(  
2      organization_id="<YOUR-ORG-ID>",  
3      delete_older_than_days=150  
4  )
```

For more information, see the [Python SDK Docs](#).

DeleteBinaryDataByFilter

Filter and delete binary data.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `filter` ([viam.proto.app.data.Filter](#)) (optional): Optional Filter specifying binary data to delete. Passing an empty Filter will lead to all data being deleted. Exercise caution when using this option. You must specify an organization ID with “organization_ids” when using this option.

Returns:

- ([int](#)): The number of items deleted.

Example:

```
1  from viam.utils import create_filter
2
3  my_filter = create_filter(component_name="left_motor", organization_ids=["<YOUR-ORG-ID>"])
4
5  res = await data_client.delete_binary_data_by_filter(my_filter)
```

For more information, see the [Python SDK Docs](#).

DeleteBinaryDataByIds

Filter and delete binary data by ids.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `binary_ids` ([List\[viam.proto.app.data.BinaryID\]](#) | [List\[str\]](#)) (required): Binary data id strings specifying the data to be deleted or BinaryID objects.

Returns:

- ([int](#)): The number of items deleted.

Raises:

- ([GRPCError](#)): If no binary data id strings or BinaryID objects are provided.

Example:

```
1  from viam.proto.app.data import BinaryID
2  from viam.utils import create_filter
3
4  my_filter = create_filter(component_name="camera-1", organization_ids=["<YOUR-ORG-ID>"])
5  binary_metadata, count, last = await data_client.binary_data_by_filter(
6      filter=my_filter,
7      limit=20,
8      include_binary_data=False
9  )
10
11  my_ids = []
12
13  for obj in binary_metadata:
14      my_ids.append(
15          obj.metadata.binary_data_id
16      )
17
18  binary_data = await data_client.delete_binary_data_by_ids(my_ids)
```

For more information, see the [Python SDK Docs](#).

AddTagsToBinaryDataByIDs

Add tags to binary data by ids.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `tags` (List[[str](#)]) (required): List of tags to add to specified binary data. Must be non-empty.
- `binary_ids` ([List\[viam.proto.app.data.BinaryID\]](#) | List[[str](#)]) (required): Binary data id strings specifying the data to be tagged or BinaryID objects. Must be non-empty. Note: BinaryID objects are deprecated and will be removed in a future release. Please use the binary data id field instead.

Returns:

- None.

Raises:

- (GRPCError): If no binary data id strings or BinaryID objects are provided.

Example:

```
1  from viam.utils import create_filter
2
3  tags = ["tag1", "tag2"]
4
5  my_filter = create_filter(component_name="camera-1", organization_ids=["<YOUR-ORG-ID>"])
6  binary_metadata, count, last = await data_client.binary_data_by_filter(
7      filter=my_filter,
8      limit=20,
9      include_binary_data=False
10 )
11
12 my_ids = []
13
14 for obj in binary_metadata:
15     my_ids.append(
16         obj.metadata.binary_data_id
17     )
18
19 binary_data = await data_client.add_tags_to_binary_data_by_ids(tags, my_ids)
```

For more information, see the [Python SDK Docs](#).

AddTagsToBinaryDataByFilter

Add tags to binary data by filter.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `tags` (List[[str](#)]) (required): List of tags to add to specified binary data. Must be non-empty.
- `filter` ([viam.proto.app.data.Filter](#)) (optional): Filter specifying binary data to tag. If no Filter is provided, all data will be tagged.

Returns:

- None.

Raises:

- (GRPCError): If no tags are provided.

Example:

```
1 from viam.utils import create_filter
2
3 my_filter = create_filter(component_name="my_camera")
4 tags = ["tag1", "tag2"]
5 await data_client.add_tags_to_binary_data_by_filter(tags, my_filter)
```

For more information, see the [Python SDK Docs](#).

RemoveTagsFromBinaryDataByIds

Remove tags from binary by ids.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `tags` (List[[str](#)]) (required): List of tags to remove from specified binary data. Must be non-empty.
- `binary_ids` ([List\[viam.proto.app.data.BinaryID\]](#) | [List\[str\]](#)) (required): Binary data id strings specifying the data to be untagged or BinaryID objects. Must be non-empty. Note: BinaryID objects are deprecated and will be removed in a future release. Please use the binary data id field instead.

Returns:

- ([int](#)): The number of tags removed.

Raises:

- ([GRPCError](#)): If no `binary_ids` or `tags` are provided.

Example:

```
1 from viam.utils import create_filter
2
3 tags = ["tag1", "tag2"]
4
5 my_filter = create_filter(component_name="camera-1")
6
7 binary_metadata, count, last = await data_client.binary_data_by_filter(
8     filter=my_filter,
9     limit=50,
10    include_binary_data=False
11 )
12
13 my_ids = []
14
15 for obj in binary_metadata:
16     my_ids.append(
17         obj.metadata.binary_data_id
18     )
19
20 binary_data = await data_client.remove_tags_from_binary_data_by_ids(
21     tags, my_ids)
```

For more information, see the [Python SDK Docs](#).

RemoveTagsFromBinaryDataByFilter

Remove tags from binary data by filter.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `tags` (List[[str](#)]) (required): List of tags to remove from specified binary data.
- `filter` ([viam.proto.app.data.Filter](#)) (optional): Filter specifying binary data to untag. If no Filter is provided, all data will be untagged.

Returns:

- ([int](#)): The number of tags removed.

Raises:

- (GRPCError): If no tags are provided.

Example:

```
1 from viam.utils import create_filter
2
3 my_filter = create_filter(component_name="my_camera")
4 tags = ["tag1", "tag2"]
5 res = await data_client.remove_tags_from_binary_data_by_filter(tags, my_filter)
```

For more information, see the [Python SDK Docs](#).

TagsByFilter

Get a list of tags using a filter.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `filter` ([viam.proto.app.data.Filter](#)) (optional): Filter specifying data to retrieve from. If no Filter is provided, all data tags will return.

Returns:

- (List[[str](#)]): The list of tags.

Example:

```
1 from viam.utils import create_filter
2
3 my_filter = create_filter(component_name="my_camera")
4 tags = await data_client.tags_by_filter(my_filter)
```

For more information, see the [Python SDK Docs](#).

AddBoundingBoxToImageById

Add a bounding box to an image specified by its BinaryID.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `binary_id` ([viam.proto.app.data.BinaryID](#) | [str](#)) (required): The binary data id or BinaryID of the image to add the bounding box to. Note: BinaryID objects are deprecated and will be removed in a future release. Please use the binary data id field instead.
- `label` ([str](#)) (required): A label for the bounding box.
- `x_min_normalized` ([float](#)) (required): Min X value of the bounding box normalized from 0 to 1.
- `y_min_normalized` ([float](#)) (required): Min Y value of the bounding box normalized from 0 to 1.
- `x_max_normalized` ([float](#)) (required): Max X value of the bounding box normalized from 0 to 1.
- `y_max_normalized` ([float](#)) (required): Max Y value of the bounding box normalized from 0 to 1.

Returns:

- ([str](#)): The bounding box ID.

Raises:

- (GRPCError): If the X or Y values are outside of the [0, 1] range.

Example:

```
1  bbox_id = await data_client.add_bounding_box_to_image_by_id(  
2      binary_id="<YOUR-BINARY-DATA-ID>",  
3      label="label",  
4      x_min_normalized=0,  
5      y_min_normalized=.1,  
6      x_max_normalized=.2,  
7      y_max_normalized=.3  
8  )  
9  
10 print(bbox_id)
```

For more information, see the [Python SDK Docs](#).

RemoveBoundingBoxFromImageById

Removes a bounding box from an image specified by its BinaryID.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `bbox_id` ([str](#)) (required): The ID of the bounding box to remove.
- `binary_id` ([viam.proto.app.data.BinaryID | str](#)) (required): The binary data id or BinaryID of the image to remove the bounding box.

Returns:

- None.

Example:

```
1  await data_client.remove_bounding_box_from_image_by_id(  
2  binary_id="<YOUR-BINARY-DATA-ID>",  
3  bbox_id="your-bounding-box-id-to-delete"  
4  )
```

For more information, see the [Python SDK Docs](#).

BoundingBoxLabelsByFilter

Get a list of bounding box labels using a Filter.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `filter` ([viam.proto.app.data.Filter](#)) (optional): Filter specifying data to retrieve from. If no Filter is provided, all labels will return.

Returns:

- (List[[str](#)]): The list of bounding box labels.

Example:

```
1 from viam.utils import create_filter
2
3 my_filter = create_filter(component_name="my_camera")
4 bounding_box_labels = await data_client.bounding_box_labels_by_filter(
5     my_filter)
6
7 print(bounding_box_labels)
```

For more information, see the [Python SDK Docs](#).

GetDatabaseConnection

Get a connection to access a MongoDB Atlas Data federation instance.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `organization_id` ([str](#)) (required): Organization to retrieve the connection for. You can obtain your organization ID from the Viam app's organization settings page.

Returns:

- ([str](#)): The hostname of the federated database.

Example:

```
1 hostname = await data_client.get_database_connection(organization_id="<YOUR-ORG-ID>")
```

For more information, see the [Python SDK Docs](#).

ConfigureDatabaseUser

Configure a database user for the Viam organization's MongoDB Atlas Data Federation instance. It can also be used to reset the password of the existing database user.

[Python](#) [TypeScript](#) [Flutter](#)

Parameters:

- `organization_id` ([str](#)) (required): The ID of the organization. You can obtain your organization ID from the Viam app's organization settings page.
- `password` ([str](#)) (required): The password of the user.

Returns:

- None.

Example:

```
1 await data_client.configure_database_user(
2     organization_id="<YOUR-ORG-ID>",
3     password="Your_Password@1234"
4 )
```

For more information, see the [Python SDK Docs](#).

AddBinaryDataToDatasetByIds

Add the `BinaryData` to the provided dataset. This BinaryData will be tagged with the `VIAM_DATASET_{id}` label.

Parameters:

- `binary_ids` ([List\[viam.proto.app.data.BinaryID\]](#) | [List\[str\]](#)) (required): The IDs of binary data to add to dataset. To retrieve these IDs, navigate to your data page, click on an image and copy its File ID from the details tab. To retrieve the dataset ID, navigate to your dataset’s page in the Viam app, and use the left-hand menu to copy the dataset ID.
- `dataset_id` ([str](#)) (required): The ID of the dataset to be added to.

Returns:

- None.

Example:

```
1  binary_metadata, count, last = await data_client.binary_data_by_filter(  
2      include_binary_data=False  
3  )  
4  
5  my_binary_ids = []  
6  
7  for obj in binary_metadata:  
8      my_binary_ids.append(  
9          obj.metadata.binary_data_id  
10     )  
11  
12  await data_client.add_binary_data_to_dataset_by_ids(  
13      binary_ids=my_binary_ids,  
14      dataset_id="abcd-1234xyz-8765z-123abc"  
15  )
```

For more information, see the [Python SDK Docs](#).

RemoveBinaryDataFromDatasetByIDs

Remove the BinaryData from the provided dataset. This BinaryData will lose the VIAM_DATASET_{id} tag.

Parameters:

- `binary_ids` ([List\[viam.proto.app.data.BinaryID\]](#) | [List\[str\]](#)) (required): The IDs of binary data to remove from dataset. To retrieve these IDs, navigate to your data page, click on an image and copy its File ID from the details tab. To retrieve the dataset ID, navigate to your dataset’s page in the Viam app, and use the left-hand menu to copy the dataset ID. Note: BinaryID objects are deprecated and will be removed in a future release. Please use the binary data id field instead.
- `dataset_id` ([str](#)) (required): The ID of the dataset to be removed from.

Returns:

- None.

Example:

```
1  binary_metadata, count, last = await data_client.binary_data_by_filter(  
2      include_binary_data=False  
3  )  
4  
5  my_binary_ids = []  
6  
7  for obj in binary_metadata:  
8      my_binary_ids.append(  
9          obj.metadata.binary_data_id  
10     )  
11  
12  await data_client.remove_binary_data_from_dataset_by_ids(  
13      binary_ids=my_binary_ids,  
14      dataset_id="abcd-1234xyz-8765z-123abc"  
15  )
```

For more information, see the [Python SDK Docs](#).

CreateDataset

Create a new dataset.

[Python](#) [Flutter](#)

Parameters:

- `name` ([str](#)) (required): The name of the dataset being created.
- `organization_id` ([str](#)) (required): The ID of the organization where the dataset is being created. You can obtain your organization ID from the Viam app's organization settings page.

Returns:

- ([str](#)): The dataset ID of the created dataset.

Example:

```
1  dataset_id = await data_client.create_dataset(  
2      name="<DATASET-NAME>",  
3      organization_id="<YOUR-ORG-ID>"  
4  )  
5  print(dataset_id)
```

For more information, see the [Python SDK Docs](#).

DeleteDataset

Delete a dataset.

[Python](#) [Flutter](#)

Parameters:

- `id` ([str](#)) (required): The ID of the dataset. You can retrieve this by navigating to the DATASETS sub-tab of the DATA tab, clicking on the dataset, clicking the ... menu and selecting Copy dataset ID.

Returns:

- None.

Example:

```
1  await data_client.delete_dataset(  
2      id="<YOUR-DATASET-ID>"  
3  )
```

For more information, see the [Python SDK Docs](#).

RenameDataset

Rename a dataset specified by the dataset ID.

[Python](#) [Flutter](#)

Parameters:

- `id` ([str](#)) (required): The ID of the dataset. You can retrieve this by navigating to the DATASETS sub-tab of the DATA tab, clicking on the dataset, clicking the ... menu and selecting Copy dataset ID.
- `name` ([str](#)) (required): The new name of the dataset.

Returns:

- None.

Example:

```
1  await data_client.rename_dataset(  
2      id="<YOUR-DATASET-ID>",  
3      name="MyDataset"  
4  )
```

For more information, see the [Python SDK Docs](#).

ListDatasetsByOrganizationID

Get the datasets in an organization.

[Python](#) [Flutter](#)

Parameters:

- `organization_id` ([str](#)) (required): The ID of the organization. You can obtain your organization ID from the Viam app's organization settings page.

Returns:

- ([Sequence\[viam.proto.app.dataset.Dataset\]](#)): The list of datasets in the organization.

Example:

```
1  datasets = await data_client.list_datasets_by_organization_id(  
2      organization_id="<YOUR-ORG-ID>"  
3  )  
4  print(datasets)
```

For more information, see the [Python SDK Docs](#).

ListDatasetsByIds

Get a list of datasets using their IDs.

[Python](#) [Flutter](#)

Parameters:

- `ids` (List[str]) (required): The IDs of the datasets being called for. To retrieve these IDs, navigate to your dataset’s page in the Viam app, click ... in the left-hand menu, and click Copy dataset ID.

Returns:

- (Sequence[viam.proto.app.dataset.Dataset]): The list of datasets.

Example:

```
1 datasets = await data_client.list_dataset_by_ids(  
2     ids=["<YOUR-DATASET-ID-1>, <YOUR-DATASET-ID-2>"]  
3 )  
4 print(datasets)
```

For more information, see the [Python SDK Docs](#).

Find part ID

To copy the ID of your machine part, select the part status dropdown to the right of your machine’s location and name on the top of its page and click the copy icon next to **Part ID**.

For example:

